

Programação Competitiva

Semana 8: Estruturas de dados avançadas

Alberto Tavares Duarte Neto

08 de Outubro, 2025

Universidade de Brasília

Definição

Um algoritmo de **programação dinâmica** é um algoritmo que resolve problemas combinando soluções de subproblemas, utilizando a memoização para evitar recalcular o mesmo subproblema.

Comecemos com um problema de motivação.

Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Algoritmo guloso 1: pegar sempre a maior quantidade de moedas funciona? Não! No exemplo abaixo, o ótimo é pegar a moeda de valor 50.



Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Algoritmo guloso 1: pegar sempre a maior quantidade de moedas funciona? Não! No exemplo abaixo, o ótimo é pegar a moeda de valor 50.



Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Algoritmo guloso 2: pegar o máximo entre a soma das moedas ímpares e a soma das moedas pares. WA!

No exemplo abaixo, o ótimo é pegar as moedas em vermelho.



Problema da Moeda

Considere $n \leq 10^5$ moedas em uma fileira com valores inteiros. Sejam $v_1, v_2, \dots, v_n$ os valores das moedas em ordem. Você deve escolher $k \leq n$ moedas de forma que:

- Nenhum par de moedas consecutivas é escolhida.
- A soma dos valores das moedas é maximizada.

Algoritmo guloso 2: pegar o máximo entre a soma das moedas ímpares e a soma das moedas pares. WA!

No exemplo abaixo, o ótimo é pegar as moedas em vermelho.



Força bruta

Vamos primeiro construir uma função recursiva f que resolve o problema com complexidade ruim.

Seja $f(i)$ a função que retorna a maior soma possível considerando apenas as moedas $i, i + 1, \dots, n - 1$ (0-indexada). Temos dois casos:

- Pegamos a moeda i ; neste caso não podemos pegar a moeda $i + 1$, então $= v_i + f(i + 1)$.
- Não pegamos a moeda i . Ainda podemos pegar a moeda $i + 1$, então $f(i) = f(i + 1)$.

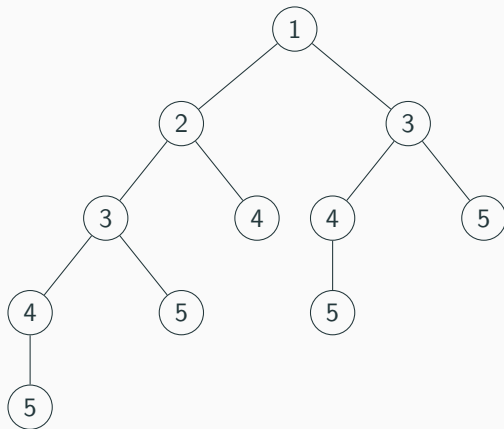
Como esses são os dois únicos casos, então

$$f(i) = \max(f(i + 1), v_i + f(i + 2)) .$$

A complexidade de f é a mesma do algoritmo que calcula fibonacci recursivamente, que é aproximadamente $O(1.6180^n)$. Isso dá TLE para $n \geq 40$.

A árvore de recursão

Analisemos a árvore de recursão da função f para $n = 4$.



Temos repetidas chamadas recursivas para $i = 3, 4, 5$. Porém, memoizando a resposta de $f(i)$ para cada i , precisamos chamar $f(i)$ uma vez para cada valor de i diferente, tornando a solução $O(n)$.

Definição

Seja f uma função recursiva. Chamaremos um elemento i do domínio da função de **estado** e chamaremos o código que faz as chamadas recursivas de **transição**.

O código da função dos slides anteriores é:

Implementação da memoização

```
int dp(int i) {  
    if(i >= n) return 0; // caso base  
    if(!memoizado[i] != -1) return tab[i];  
    memoizado[i] = 1;  
    return tab[i] = max(dp(i+2), v[i] + dp(i+1));  
}
```

A complexidade de uma DP é a soma da complexidade da transição de cada estado. Como a transição do código acima é sempre constante, então a complexidade é $O(\text{estados}) = O(n)$.

Resumo da DP da moeda

Resumo do processo:

- Escolha estados que representem seu problema.
- Escreva uma função que resolva o problema a partir de subproblemas.
- Calcule a complexidade. Se a transição de cada estado tem a mesma complexidade, então a complexidade total é $O(\text{estados} * \text{transicao})$.

No problema anterior, decidimos escolher as moedas da esquerda para a direita. Então o estado deve ser $E = \{0, 1, \dots, n\}$, onde $i \in E$ representa que ainda temos as moedas $i, i + 1, \dots, n - 1$ para escolher.

A função recursiva que resolve o problema é

$$\text{dp}(i) = \max(\text{dp}(i + 1), v_i + \text{dp}(i + 2)) .$$

.

Como a transição de todo estado é constante, a complexidade é $O(n)$.

Problema da mochila

Você tem $n \leq 1000$ objetos e uma mochila com capacidade $c \leq 1000$. Cada objeto tem um valor v_i e um peso p_i . Você deve escolher objetos para colocar na mochila satisfazendo:

- A soma dos pesos dos objetos escolhidos não deve ultrapassar c .
- A soma dos valores dos objetos escolhidos deve ser maximizada.

Antes de passar para o próximo slide, tente escolher os estados e escreva a recorrência para que o código seja executado em menos de 1 segundo.

Escolheremos os objetos da esquerda para a direita. Definimos $dp(i, p)$ onde i representa que podemos escolher as moedas $i, i + 1, \dots, n - 1$ e p representa o peso dos objetos escolhidos até então.

Testando as possibilidades de pegar ou não o objeto i , a transição é definida pela fórmula

$$dp(i, k) = \max(dp(i + 1, k), v_i + dp(i + 1, k + p_i)) .$$

Problema da mochila

Você tem $n \leq 1000$ objetos e uma mochila com capacidade $c \leq 1000$. Cada objeto tem um valor v_i e um peso p_i . Você deve escolher objetos para colocar na mochila satisfazendo:

- A soma dos pesos dos objetos escolhidos não deve ultrapassar c .
- A soma dos valores dos objetos escolhidos deve ser maximizada.

Antes de passar para o próximo slide, tente escolher os estados e escreva a recorrência para que o código seja executado em menos de 1 segundo.

Escolheremos os objetos da esquerda para a direita. Definimos $dp(i, p)$ onde i representa que podemos escolher as moedas $i, i + 1, \dots, n - 1$ e p representa o peso dos objetos escolhidos até então.

Testando as possibilidades de pegar ou não o objeto i , a transição é definida pela fórmula

$$dp(i, k) = \max(dp(i + 1, k), v_i + dp(i + 1, k + p_i)) .$$

Problema da maior subsequência crescente (LIS)

Dado um vetor com $n \leq 1000$ elementos, encontre sua maior subsequência crescente.

Exemplo:

Problema da maior subsequência crescente (LIS)

Dado um vetor com $n \leq 1000$ elementos, encontre sua maior subsequência crescente.

Exemplo:



Problema da maior subsequência crescente (LIS)

Dado um vetor com $n \leq 1000$ elementos, encontre sua maior subsequência crescente.

Exemplo:



Uma subsequência crescente com maior comprimento possível é (2, 4, 6, 9) (em vermelho).

Vamos definir uma função $dp(i)$ como sendo a maior subsequência começando na posição i . Como definimos $dp(i)$ recursivamente, isto é, a partir de $dp(j)$ para $i < j$?

$$dp(i) = \max_{i < j \text{ e } v_i < v_j} (dp(j) + 1) .$$

Implementação da LIS quadrática

```
vector<int> dp(n, 0);  
for(int i = n-1; i >= 0; i--) {  
    dp[i] = 1;  
    for(int j = n-1; j > i; j--) {  
        if(v[i] < v[j]) {  
            dp[i] = max(dp[i], dp[j]+1);  
        }  
    }  
}
```

$$dp(i) = \max_{i < j \text{ e } v_i < v_j} (dp(j) + 1) .$$

Complexidade: $O(n^2)$.

Notemos na fórmula da recursão

$$dp(i) = \max_{i < j \text{ e } v_i < v_j} (dp(j) + 1)$$

que estamos tomando o máximo entre os valores $dp(j)$ tais que $v_j \in [v_i + 1, \text{MAX}]$. Fazemos isso com um for, que deixa a solução quadrática. Como podemos otimizar isso?

Vamos criar uma Segtree que calcula máximo de intervalo, e armazenar o valor $dp(j)$ na posição v_j da Segtree. Se houver mais de uma posição com mesmo valor, armazenamos o maior deles.

No lugar do for, faremos uma consulta de máximo no intervalo $[v_i + 1, \text{MAX}]$.

Vamos assumir que $v_i \leq n$ para todo i .

Implementação da LIS otimizada

```
ST seg(); // segtree que calcula maximo e atualiza  $v[i] = val$ 
vector<int> dp(n, 0);
for(int i = n-1; i >= 0; i--) {
    dp[i] = 1;
    dp[i] = max(dp[i], seg.query(v[i]+1, n));
    int valor_antigo = seg.query(v[i], v[i]);
    if(dp[i] > valor_antigo) seg.update(v[i], dp[i]);
}
```

Complexidade: $O(n \log n)$.

Observação: se $v_i \leq 10^9$, a priori não será possível declarar a Segtree. Podemos comprimir os valores:

Compressão de vetor

Observação: se $v_i \leq 10^9$, a priori não será possível declarar a Segtree. Podemos comprimir os valores do vetor antes de construir a Segtree:

Compressão

```
vector<int> vals = v;
sort(vals.begin(), vals.end()); // ordena
vals.erase(unique(vals.begin(), vals.end()), vals.end());

map<int,int> traducao;
for(int i = 0; i < (int)vals.size(); i++)
    traducao[v[i]] = i; // agora todo valor esta entre 0 e n-1

for(int i = 0; i < n; i++) {
    v[i] = traducao[v[i]];
}
```

Essa é uma técnica comum para problemas nos quais os valores em si não importam e usam esse tipo de estrutura de dados.